OpenSpot

Sprint Planning

Team Members:

- Sayed
- Khalil
- Jack
- Anthony
- Dhwani
- Lailani

OpenSpot Sprint Roadmap

Sprint 1 – Foundation

Sprint 2 – Stability

Sprint 3 – Expansion

Sprint 4 – Optimization & Demo Readiness

Sprint 1 - Database Developer

🎯 Sprint 1 Goal

Build the Supabase database foundation so the backend can function properly.

🛠️ What I'm Setting Up:

- Design the ER Structure
- Create core tables in Supabase
  - Garage
  - Sensors
  - Sensor_events
  - Garage_state
- Insert Seed Data
  - 1 garage
  - 2 sensors (Entry + Exit)

Sprint 1 Database Continued

🔒 Add Data Validation Rules

- No negative counts
- Cannot exceed garage capacity
- Direction limited to IN / OUT

🧪 Testing & Verification

- Test basic counting logic
- Confirm updates reflect correctly
- Ensure database is ready for backend integration

✅ Sprint 1 Outcome

A stable, validated database ready for real-time sensor ingestion.

Further Sprint Planning - Database Developer

Sprint 2 – Stability & Performance

- Add indexes to improve query speed
- Test concurrent sensor updates
- Verify accurate real-time counting

Sprint 3 – Backend & Real-Time Integration

- Connect backend fully to the Database
- Implement safe transaction-based count updates
- Enable real-time dashboard updates

Sprint 4 – Optimization & Finalization

- Optimize performance under higher load
- Support advanced analytics (trends, peak usage)
- Finalize documentation and prepare for deployment

Sprint 1 - Backend Developer

Goal:

Establish a backend infrastructure and process garage exit/entry events using FastAPI and Supabase

Completed in Sprint 1:

- Initialized FastAPI project structure
- Connected backend to Supabase
- Implemented POST /garage-event endpoint
- Implemented GET /status endpoint
- Added basic validation and error handling
- Enable API documentation

Sprint 1 - Backend Developer

Overall System Flow:

Sensors:

- FastAPI Backed
- Supabase Database
- GET /status
- Frontend Dashboard

Sprint 1 Impact:

- Established core data pipeline
- Enable accurate counting for Sprint 2
- Prepared system for real-time updates for Sprint 3

Dependency:

- Supabase connection points to the database
- Frontend to display the data from the database and sensors

Further Sprint Planning - Backend Developer

Completed in Sprint 2:

- Implement accurate occupied spot calculation with capacity enforcement
- Improve ingestion reliability and prevent invalid or duplicate events

Completed in Sprint 3:

- Integrate real entrance/exit sensors and replace simulated events
- Implement real-time updates and safely handle rapid consecutive events

Completed in Sprint 4:

- Improve logging, error handling, and overall system stability
- Optimize performance and finalize API documentation with stress testing

Sprint 1-Hardware Systems

Detection Prototype & Integration

Goal:

Build and validate a dual-ultrasonic ESP32 system that :

- Detects vehicle direction using staggered sensors
- Maintains an accurate local spot counter
- Prevents false triggers through state based logic
- Transmits real time entry/exit events to the backend via wifi

Sprint 1-Hardware Systems

Detection Prototype & Integration

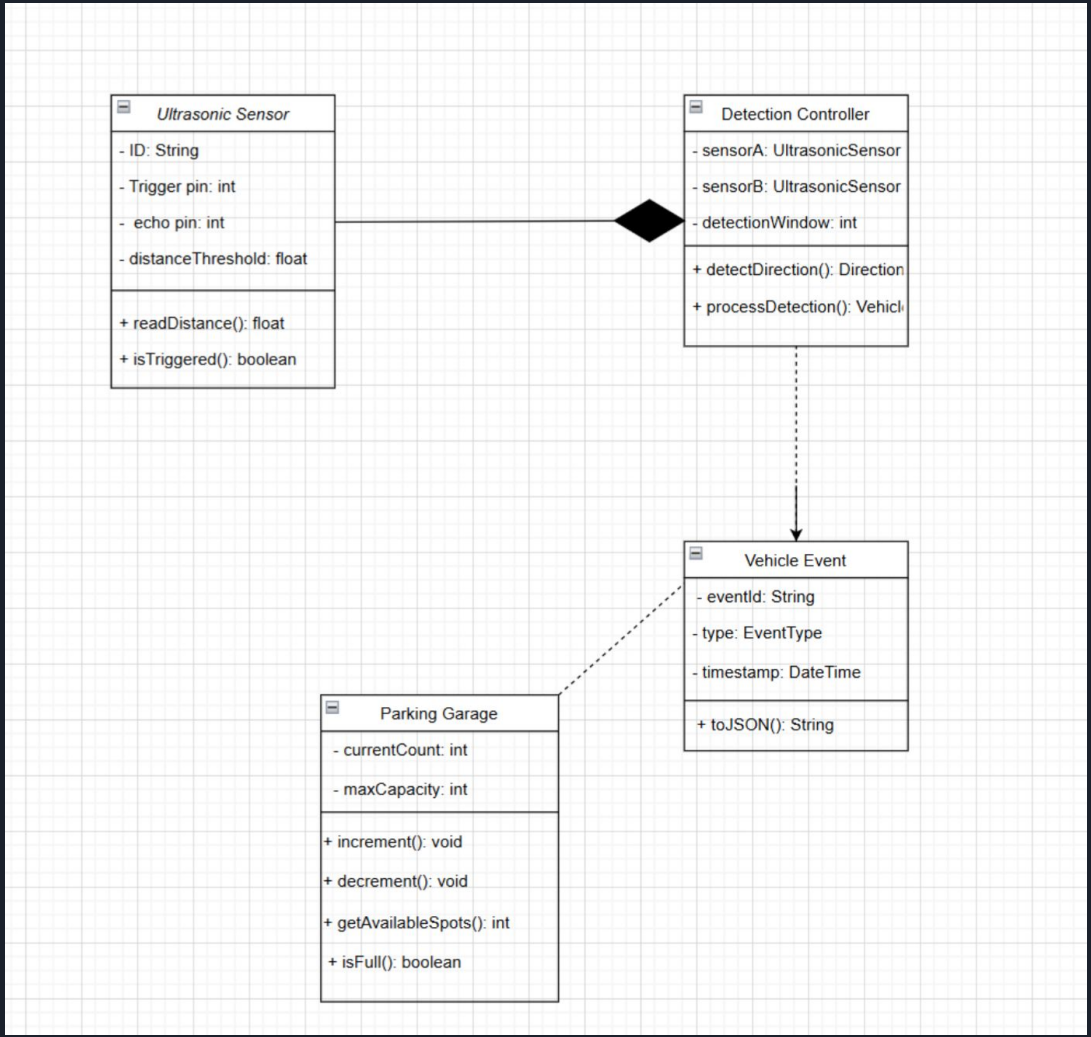
Hardware + Logic

- ESP32 with Ultrasonic sensor A+B(staggered)
- Sequence based detection:
  A→B = Entry
  B→A = Exit
- State machine to prevent false triggers
- Local spot counter(+/-)

Effort Estimation (Hardware – Sprint 1)

Effort Level: Medium–High

Hardware setup and calibration require iterative testing to ensure accurate directional detection and prevent false triggers. Additional complexity comes from implementing reliable WiFi communication and validating end-to-end integration with the backend.


Ultrasonic Sensor
- ID: String
- Trigger pin: int
- echo pin: int
- distanceThreshold: float
+ readDistance(): float
+ isTriggered(): boolean
Detection Controller
- sensorA: UltrasonicSensor
- sensorB: UltrasonicSensor
- detectionWindow: int
+ detectDirection(): Direction
+ processDetection(): Vehicl
Vehicle Event
- eventId: String
- type: EventType
- timestamp: DateTime
+ toJSON(): String
Parking Garage
- currentCount: int
- maxCapacity: int
+ increment(): void
+ decrement(): void
+ getAvailableSpots(): int
+ isFull(): boolean

Further Sprint Planning - Hardware

Completed in Sprint 2:

- Implement reliable directional detection (A→B entry / B→A exit)
- Finalize accurate local spot counting logic

Completed in Sprint 3:

- Integrate solar power system (battery + panel)
- Design, 3D print, and assemble enclosure
- Mount prototype in test environment

Completed in Sprint 4:

- Optimize detection accuracy and reduce false triggers
- Finalize mounting solution for garage deployment
- Validate multi-node hardware performance under stress testing

Sprint 1-Frontend Systems

Foundation + Data Integration

Goal:

Build the frontend foundation for OpenSpot by developing the core UI and displaying real-time parking availability using API integration or mock data:

- Set up React project structure
- Create folder organization + styling
- Build main dashboard(total spots / available spots)
- Implement different garage views
- Basic Navigation + UI styling

Sprint 1-Frontend Systems

Foundation + Data Integration

Data Integration:

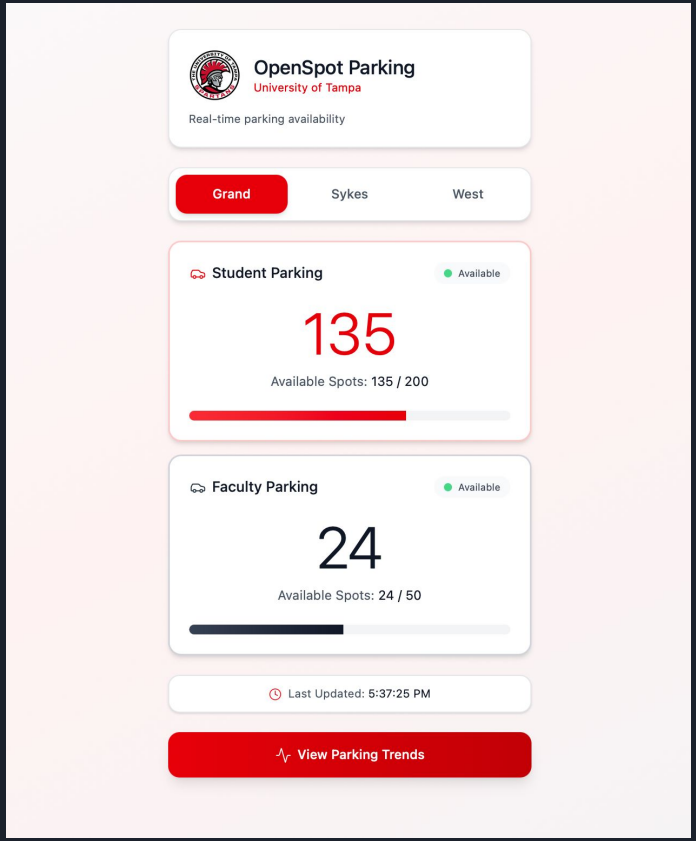
- Connect to backend API (mock data at first)
- Display live availability updates
- Prepare for supabase integration

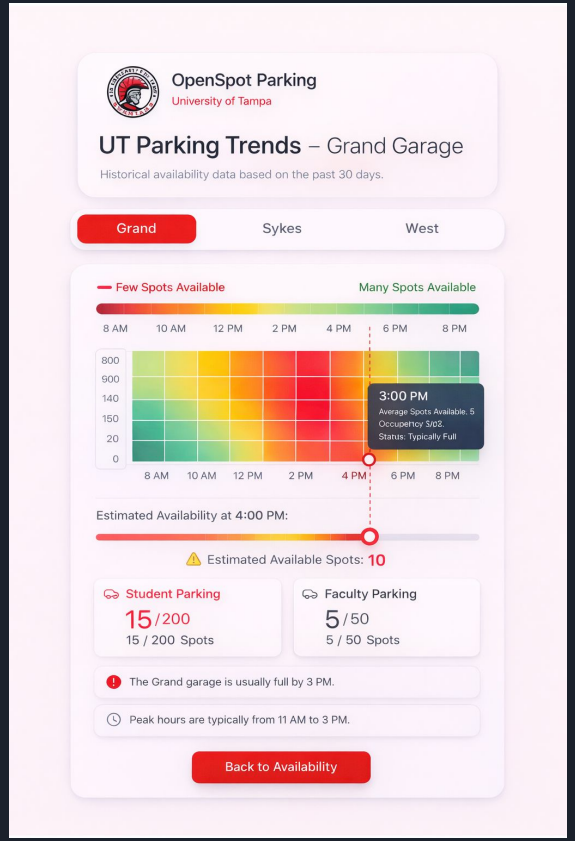
Effort Estimation:

Low-medium.

Core UI can be completed within 1-2 weeks. Main dependency is backend readiness for full integration

UI Mockups


OpenSpot Parking
University of Tampa
Real-time parking availability
Grand
Sykes
West
Student Parking
Available
135
Available Spots: 135 / 200
Faculty Parking
Available
24
Available Spots: 24 / 50
Last Updated: 5:37:25 PM
View Parking Trends


OpenSpot Parking
University of Tampa
UT Parking Trends – Grand Garage
Historical availability data based on the past 30 days.
Grand
Sykes
West
Few Spots Available
Many Spots Available
8 AM
10 AM
12 PM
2 PM
4 PM
6 PM
8 PM
800
900
140
150
20
0
3:00 PM
Average Spots Available: 5
Occupancy 5%
Status: Typically Full
Estimated Availability at 4:00 PM:
Estimated Available Spots: 10
Student Parking
15 / 200
15 / 200 Spots
Faculty Parking
5 / 50
5 / 50 Spots
The Grand garage is usually full by 3 PM.
Peak hours are typically from 11 AM to 3 PM.
Back to Availability

Trello

Search

Create

Senior Capstone

J AE JG KS LG +2

Share

Product Backlog

Add a card

To do / Sprint

Hardware Sprint1:

J LG

Backend Sprint1:

KS

Frontend Sprint1:

SA

Database Sprint1:

AE

Add a card

In Progress

Priority and Order

KS AE D J LG SA

Add a card

Testing/ Review

Add a card

Done

Add a card

Inbox Planner Board Switch boards

To do / Sprint

Hardware Sprint1:

+ Add | Dates | Checklist | Attachment

Members: J, LG

Labels: Effort: Medium

Description Edit

Sprint 1:

Build and validate a dual-ultrasonic ESP32 system that :

- Detects vehicle direction using staggered sensors
- Maintains an accurate local spot counter
- Prevents false triggers through state based logic
- Transmits real time entry/exit events to the backend via wifi

To do / Sprint

Backend Sprint1:

+ Add | Dates | Checklist | Attachment

Members: KS

Labels: Effort: Medium

Description Edit

Goal:

Establish a backend infrastructure and process garage exit/entry events using FastAPI and Supabase

- Initialized FastAPI project structure
- Connected backend to Supabase
- Implemented POST /garage-event endpoint
- Implemented GET /status endpoint
- Added basic validation and error handling
- Enable API documentation

To do / Sprint

Frontend Sprint1:

+ Add | Dates | Checklist | Attachment

Members: SA

Labels: Effort: Medium

Description Edit

Build the frontend foundation for OpenSpot by developing the core UI and displaying real-time parking availability using API integration or mock data:

- Set up React project structure
- Create folder organization + styling
- Build main dashboard(total spots / available spots)
- Implement different garage views
- Basic Navigation + UI styling

To do / Sprint

Database Sprint1:

+ Add | Dates | Checklist | Attachment

Members: AE

Labels: Effort: Medium

Description Edit

Build Supabase Database foundation so the backend can function properly.

- Test basic counting logic
- Confirm updates reflect correctly
- Ensure database is ready for backend integration

Labels

Search labels...

Labels

- [ ] Effort Easy
- [ ] Effort: Hard
- [x] Effort: Medium

Create a new label

Enable colorblind friendly mode

In Progress

Priority and Order

+ Add | Labels | Dates | Checklist | Attachment

Members: KS, AE, D, J, LG, SA

Description Edit

Database Priority: Anthony

- Design ER structure
- Create tables: `garage`, `sensors`, `sensor_events`, `garage_state`
- Add constraints (no negative counts, no capacity overflow, IN/OUT only)
- Insert seed data (1 garage, 2 sensors)
- Add indexes and test concurrent updates
- Document schema

Backend Priority: Khalil

- Start FastAPI locally
- Configure Supabase connection
- Implement `POST /garage-event`
- Log events in database
- Implement `GET /status` returning totals
- Add basic validation and error handling

Hardware Priority: Jack + Lailani

- Wire ESP32 + ultrasonic sensors
- Implement 5V → 3.3V echo pin protection